

AI-AUGMENTED BACKEND DEVELOPMENT

AI BlogNest API

3. Project Design Phase

Architecture, MVC structure, database design, API design, AI flow, and security design

1. Design Objective

The project design phase defines how requirements are translated into a modular backend architecture. The sample describes a layered server model in which incoming requests pass through filtering and application layers before reaching database services. ■filecite■turn0file0■L65-L82■

2. Overall Architecture

- API/interface layer for tools such as Postman.
- Express server gateway for HTTP methods, JSON parsing, ports, and CORS.
- Authentication and role-matrix middleware for token validation and authorization.
- Granular service modules for blog lifecycle, comments, and reporting.
- Database interface layer using Mongoose to communicate with MongoDB.
- AI flow service for automated generation and summarization.

3. MVC Design

The backend follows an MVC-oriented separation. The Model layer defines Mongoose schemas and data rules. The Controller layer processes requests, validates constraints, delegates business logic, and returns JSON responses. Because the application is a headless API, the conventional View layer is represented by the API routing interface rather than HTML templates. ■filecite■turn0file0■L136-L152■

4. Data Design

- User entity: `_id`, name, email, hashed password, and role.
- Post entity: `_id`, `userID`, title/name, photo path, message/content, likes, and timestamps.
- User-to-Post relationship: one user can submit multiple blog documents.
- User-to-Comment relationship: users can create multiple comments.
- Post-to-Comment relationship: a post can contain multiple comment submissions.

5. API Design

- GET routes retrieve resources.
- POST routes create users, authentication sessions, blogs, comments, or AI requests.
- PUT routes modify existing resources or workflow states.
- DELETE routes remove resources where the role permits.

- Protected routes apply JWT and role checks before sensitive operations.

6. AI Service Design

The AI Flow service is designed as a separate integration layer. It exposes specialized endpoints such as `/api/ai/generate-blog` and `/api/ai/summarize` so controllers can delegate complex text processing to an external AI model while keeping the main backend architecture modular. ■filecite■turn0file0■L79-L85■

7. Security Design

- JWT tokens for stateless authentication.
- `bcrypt.js` for password hashing.
- Role-based route guarding.
- Input validation and sanitization.
- Protection against XSS and NoSQL parameter manipulation.
- Rate limiting and structured logging.